

ADR-001 — Neo4j over a Relational Database

Status: Accepted

Date: 2026-07-06

Context

The project models a student social network where the core entities — students, clubs, and events — are heavily interconnected. The two main analytical tasks are community detection and link prediction, both of which require traversing multi-hop relationships at query time (e.g. friends of friends, co-membership paths, participation overlap). We need to choose a storage engine that can support these traversals efficiently and natively.

Decision

We will use **Neo4j** as the primary database instead of a relational database (PostgreSQL, MySQL, etc.).

Reasons

Relationship traversal performance. In a relational database, finding friends of friends requires multiple self-joins on a large adjacency table. As the depth of traversal increases, the number of joins grows exponentially and query performance degrades. Neo4j stores relationships as first-class pointers between nodes, making multi-hop traversals constant-time per hop regardless of graph size.

Native graph algorithms. Neo4j ships with the Graph Data Science (GDS) plugin, which provides production-ready implementations of Louvain, Leiden, Label Propagation,

PageRank, Node2Vec, and similarity measures. Using a relational database would require extracting the entire adjacency matrix into Python and running algorithms externally, adding complexity and memory overhead.

Schema flexibility. Relationship properties (e.g. `role` on `MEMBRE_DE`, `since` on `EST_AMI_DE`) are naturally modeled on edges in Neo4j. In a relational model, these would require junction tables with additional foreign keys, increasing schema complexity and join depth.

Cypher readability. Graph queries expressed in Cypher are significantly more readable and maintainable than equivalent SQL with multiple self-joins, which reduces the risk of query logic errors during development.

Consequences

- The development environment requires running a Neo4j instance (managed via Docker).
- The team must learn Cypher in addition to standard query languages — allocated as GDS-10 and GDS-11 in Sprint 1.
- Data import requires the official `neo4j` Python driver instead of a standard ORM.
- All graph algorithm execution happens inside Neo4j GDS, which reduces the need for large in-memory data structures in Python.